Public Technical Brief • 2026-10

Standing on KVMem's Shoulders

Async Page Pipeline: from “fits in” to “never stalls” — exploration and practice

Thanks to the KVMem authors:Di Chai · Leye Wang · Zeshen Su · Zhiguo Xia · Zhihang Yu (arXiv 2609.04852)

KV virtualization
Pioneers

Async Page Pipeline
Us

Keep it fast
Goal

THANKS FIRST

First: a tribute to the original authors' pioneering work

With no precedent to follow, they answered a question once deemed impossible:

“Can a consumer 16 GB GPU carry a million-token agent workspace?”

Paper & innovation

KVMem: Virtualizing Million-Token Agent Workspaces on a Consumer GPU (arXiv 2609.04852 · Bilibili: FirePKU)

Open-source engineering

kvmem/kvmem: dual backends formalized (llama.cpp + NInfer)
Windows RTX 50 official runtime bundle
full acceptance & failure records

Ecosystem lineage

Shensanshu — first to combine NInfer × KVMem
tancau — Ring implementation · direct discussions
Shuaqi-de-zxz — 5060Ti line · fusion plan
llama.cpp · Neroued/ninfer · iamwavecut/ninfer-all

Our line starts from Shensanshu's NInfer × KVMem combination; thanks to tancau and Shuaqi-de-zxz for direct exchanges; recently KVMem upstream also adopted NInfer as an official backend — technical convergence.

THEIR INSIGHT

The original insight: each step only needs to see a small part

Agent workloads push context to hundreds of thousands — even a million — tokens (1M-token KV ≈ 32 GiB; consumer cards have 16 GB) — yet each step truly “sees” only a small part of the KV:

Sink (fixed head)

e.g. 12K tokens · resident

Recent (recent window)

e.g. 1K tokens · resident

All remaining history

swapped in only when retrieval hits

→ So: full KV lives in host RAM (optionally tiered to SSD); the GPU keeps only “what is currently needed” — KV virtualization is born

THEIR WORK

Pioneering techniques & public progress

First-of-their-kind, systematically practiced

- KV virtualization (quant scales + MTP KV tiered together)
- retrieval-driven on-demand swap-in (not full moves)
- LRFU history weighting + GQA-max scoring
- Ownership move (restore without copying or requantizing)
- Quota + versioned publish + read lease
- Unified cross-backend memory contract (shared by both engines)

Public progress (0.18.0 / RC3)

Dual backends formalized

llama.cpp + NInfer; Windows RTX 50 (sm120a) official bundle

RC3 measurements (5060 Ti 16GB + IQ3_S)

long-history tool turn 53.24 tok/s · 260K-token input

daily text 50.08 / code 64.54 tok/s

Peak full-card VRAM ≈ 14.12 GiB

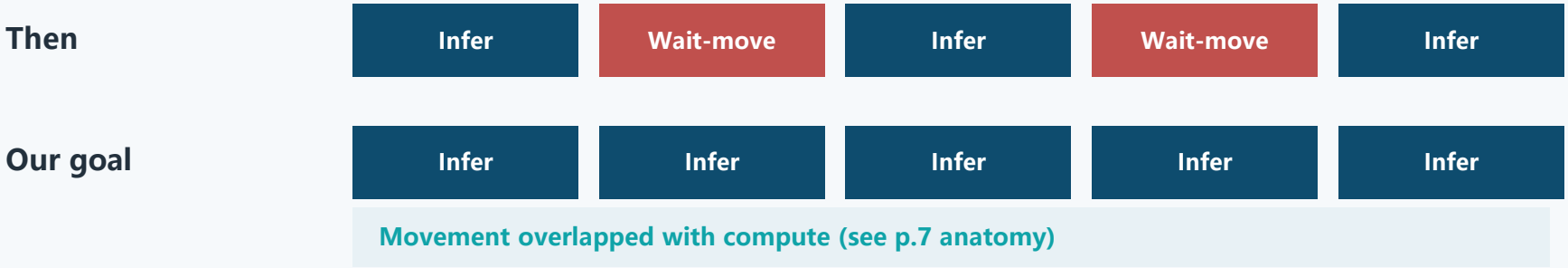
Engineering discipline

Paired statistics (95% upper bound) · boundaries "explicitly refused rather than guessed"

full acceptance & failure records public

THE WALL

The wall left behind: synchronous paging



Corollary of synchronous paging:the engineering red line "pool must not be smaller than the working set" is, in essence,**avoiding paging**— if the pool covers demand, no paging happens. But real agent context keeps growing; avoidance always ends.

Our exploration starts at this wall: turning "avoiding paging" into "mastering paging".

OUR EXPLORATION ①

Async Page Pipeline: movement and compute, at the same time



Pages + batches

64 tokens/page; 8 pages per batch
bounds each move

Submit & return

decode never waits
the essence of “async”

In-flight counting

the “left device, not yet published”
intermediate state, honestly accounted

publish / drop

publish only after verification
drop only when useless

ANATOMY

How is the movement “hidden”?

Chef

GPU compute units (SM)

runs the entire decode graph
~27-32 ms per decode step

Dedicated runner

GPU copy engine (DMA)

moves pages between VRAM and host RAM
~1-3 ms per 8-page batch

Manager

host scheduler thread

three filters: protection → watermark → relevance scoring
microseconds — never moves anything itself

Not “the chef stops cooking to fetch ingredients” — the kitchen keeps cooking while a dedicated runner shuttles between pantry and stove. Two pairs of hands, at once.

1-3 ms hidden inside 27-32 ms → fully hidden | 74.4K deep over-pool → 114.4 tok/s

Double insurance: better decisions (protection / watermark / scoring) + cheaper mistakes (async hiding + A2 lossless); in-flight counting + ledger guarantee no loss, no duplication.

OUR EXPLORATION ②

Objective fixed: the pool turns from a “capacity gate” into a “knob”

After async, the pool's benefit curve shows a three-segment structure (200K context):

Pool < working set (~42K)	Sweet spot 42K-56K	Preferred 56K-96K	> 96K (above request line)
moving every step — carried by the pipeline	zero movement in normal turns	covers most sessions	marginal gain collapses

Measured:64K vs 51K pool: only ~8 more turns — beyond the working-set line, extra pool mainly guards deep pile-ups, and the pipeline already covers those.

16 GB VRAM is zero-sum: every GiB given to the pool is one GiB taken from KV precision or weight tier — and precision affects every turn. → Shallow pool + deep pipeline + high precision.

OUR EXPLORATION ③

Correctness engineering: three rules for the “runner”

Async = we hired a runner (see p.7). Bad rules mean lost or misplaced packages — so we gave him three rules, plus an “exam paper”:

Rule 1: never move a book that is in use: the engine's “blueprint” is fixed at startup (capture phase); no runtime changes — to change, restart. Boundary crystal clear.

Rule 2: keep the old copy until the new one lands: before a new replica is published, the old one is never dropped — a “sole valid” copy is always at hand.

Rule 3: a cancelled order still finishes its run: requests can cancel, but in-flight moves must drain before accounting — no “vanished mid-way” accounts.

That “exam paper” (lossless gate A2):the same question answered via the “in-pool” and “first over-pool” paths**byte-identical**— async changes only the timing of movement, never the model output.

Honest disclosure:at slow peaks the movement “can't hide” (a batch approaches the step window) — the pipeline's breakdown boundary; the fix is a deeper pipeline (see issues page).

STATUS

Achieved so far: early results at V2 launch (2026-10-11~14)

Item	Status
Page-level async pipeline (move ‖ decode)	delivered & promoted to default
Pool calibration	81,920 tokens measured (16 GB card, low-bit KV tier)
Speed	peak 400+ tok/s; 1-hour average 220 tok/s
Deep over-pool	74.4K context (pool ~65K): 114.4 tok/s
Losslessness	in-pool vs over-pool byte-identical (A2 gate)
Scope	single-stream long-context agent sessions

↑ Early results at V2 launch (day one: peak 333, over-pool 103-195 tok/s; table completed over the following days) — NOT the latest state; latest progress (valley manhunt / deep-turn AB proof) on pages 13-14.

Stage: phase 1 (feasibility & stability) complete — from "it runs" to "runs long without collapsing, byte-lossless"; entering phase 2 (performance deep-dive).

MILESTONES

Milestones: the one-day birth of the async pipeline

Dawn • problem defined	page drift: after 34 turns, 330 pages stranded on host — decode falls to 3 tok/s
14:35-17:39 • three builds	cancel-checkpoint first → batched moves (8 pages) → root cause found: synchronous D2H copy (bimodal 10-20×)
20:36 • async move shipped	transfer-stream async submit + harvest-next-step (~600 lines) — first battle: peak 333 tok/s
Night • verification loop	over-pool decode 103-195 tok/s (was 4-6) — 20-40×; 10+ turns with zero slowdown
10-13 • promoted	independent cross-review (2×40 deep rounds) + real long sessions → default baseline: 127 turns, 0×500, peak 403.7 tok/s

One day: "define → build → localize → ship → verify"; every step carries an A/B rollback arm (NINFER_ASYNC_DEMOTE=0).

OPEN QUEST

An open problem: a transient slow state (the “death valley”)

Symptoms & fingerprint

- sporadic latency spikes: one step can go from ~30 ms to hundreds of ms (seconds at the deep end)
- fingerprint: GPU fully utilized / at full clocks, yet power draw is tiny and VRAM barely moves
- self-heals in minutes; output byte-lossless (identical to the in-pool path)
- once reproducible only via “hour-scale windows” — now manufacturable on demand (right)

Progress (latest)

- on-demand reproduction: external VRAM pressure (~1.2 GiB) ⇒ valley on pressure (speed /25), recovery on release, repeatable
- localized: all time in host-side submit calls (submit≈wall, wait=sync≈0) ⇒ driver-internal blocking
- ruled out: PDL ✕ (former prime suspect) · page bounce ✕ · host paths ✕ · page tiers ✕ · pool size ✕ · throttling ✕
- mitigation: avoid VRAM contention (the only confirmed one); minimal repro protocol ready (for upstream)

Mechanism localized: the whole round's time is spent in host-side submit calls (submit ≈ wall, wait = sync ≈ 0) ⇒ the blockage is in the engine's own submission path (driver-internal blocking) —external compute / copies / throttling all normal.

Milestone: on-demand reproduction (~1.2 GiB external VRAM pressure); a 3-step, ~2-minute minimal repro protocol is ready for upstream / driver-side analysis. Still open: the natural trigger.

MANHUNT

Manhunt in progress: no culprit yet, but the circle is closing

The hunt has run **120 hours (5 days & nights), nonstop** (10-11 → 10-16): engineering-grade controlled stress testing + field forensics throughout; numerous issues fixed and improved along the way (instrument defects, regressions, self-inflicted bugs). Today (10-16) we ran another deep over-pool AB test of the async pipeline — results on the next page.

The async pipeline's contribution: same-condition proof

- deep multi-turn (2.3-3.5× over-pool), async removed: decode ~20% slower (r=0.80)
- deepest segment (3.5× over-pool): ≈ 2.0× gap (235.6 vs 119.4 tok/s)
- ring stalls: 33 → 2 (~16×)
- shallow / one-shot: a tie (r=1.13) ⇒ the value "depends on working-set depth"

Suspect range: an ever-tightening process of elimination

- ruled out: PCIe (formerly prime suspect) · page bounce · host paths · page tiers · pool size · throttling — each measured
- locked on: all time in host-side submit calls (submit≈wall) ⇒ driver-internal blocking
- now reproducible on demand (external VRAM pressure): no more "waiting for windows"
- remaining suspects: inside the driver submit path · the natural trigger · the superposition hypothesis

No culprit yet — but the hunt has shifted from "waiting for it" to "manufacturing it on demand" — what remains: pin the exact blocking point to line level (driver-side localization + field forensics in parallel).

Note: evidence basis = n=3/arm, single machine, healthy period; valley-period impact not covered (future work).

EVIDENCE

The verdict experiment: deep over-pool AB ablation (2026-10-16)

Arm	run	round1 (1.15× over-pool)	round2 (2.3×)	round3 (3.5×)	deep mean(r2,r3)
OFF (sync fallback)	1	327.6	218.9	228.1	223.5
OFF	2	332.0	191.5	107.5	149.5
OFF	3	325.8	256.7	119.4	188.1
ON (async)	1	293.9	285.7	237.3	261.5
ON	2	303.1	264.2	125.8	195.0
ON	3	330.1	234.2	235.6	234.9

Setup: context ~75.3K → ~152K → ~229K tokens (pool 65,536 ⇒ 1.15× / 2.3× / 3.5× over-pool); long outputs up to 2000 tokens; partial prefix-cache reuse between turns (49.3% / 66.4%); the two arms differ only by the NINFER_ASYNC_DEMOTE switch

Primary criterion: OFF deep-segment median 188.1 vs ON 234.9 ⇒ r = 0.80 < 0.85: net async gain confirmed; ON wins all three paired rounds

Deepest segment (3.5× over-pool): 119.4 vs 235.6 ≈ 2.0×; ring stalls 33 → 2 (~16×)

Shallow control (one-shot, same day): r = 1.13, a tie ⇒ the value grows with working-set depth — async was designed exactly for deep loads

Honest note: n=3/arm, single machine, healthy period; the sync arm did not "die" — the stronger claim "multi-turn = death before v2" did not reproduce (we observe "usable but slow"); **and no valley appeared this time — possibly because earlier rounds of fixes incidentally eased that state** (not reproduced in this ablation; pending further verification).

OPEN ISSUES

Remaining issues (an honest list)

Pipeline breakdown boundary

at slow peaks a batch takes 11.5-33.7 ms, approaching/exceeding the decode step window — inherent exposure of any async scheme

Deep pipeline not default yet

multiple batches in flight is the right direction to dilute breakdown; under validation

SSD tier: a deliberate trade-off

full KV fits in host RAM (worst case ≈ 8.6 GB vs 128 GB); no capacity need; the +11% hot cost is not worth it

Copy-based movement

still copy-based; upstream already ships "ownership move" — catching up. Once landed: copy-vs-move contrast under the on-demand protocol (valley rate/depth) and its effect on exposure.

Coverage gap

we focus on single-stream long context; multi-session concurrency (upstream's home turf) is under-covered

Note: the "transient slow state" is expanded on the previous page.

ROADMAP

Roadmap

1	**Deeper pipeline**	multiple batches in flight, reclaiming breakdown headroom — the final link of "shallow pool + deep pipeline + high precision"
2	**Ownership move**	aligning with upstream semantics: move without copying, restore with zero snapshots
3	**Host-tier deepening**	expand host quota + optimize RAM↔VRAM movement; SSD re-evaluated on demand (not a follow item)
4	**Closing the slow state**	upstream localization of the driver submit blockage (3-step repro ready); chasing the natural trigger
5	**Mis-move rate observability**	measure the "moved-out, quickly moved-back" ratio — turning decision quality from guesswork into numbers
6	**Upstream contribution**	contribute fixes and measurements back (incl. convergence with KVMem's official backend)

Why open-source this work

Origin: the time has come to open the door

- the tech has reached "production-usable in a bounded scenario": single-stream long-context agent sessions (exclusive VRAM), verified in real runs
- but the biggest open problem (the transient slow state) exceeds what one line of effort can carry —
- **open source = community / upstream / driver-side co-investigation**
- we hold the key to "exchanging for help": a ~2-minute minimal repro protocol (the state can be manufactured at will)

Positioning & promises (definition first)

- positioning: "research-grade engineering + production-usable in a bounded scenario"; multi-session not covered
- Promise ① valley is manageable: fingerprint detectable / minutes to self-heal / confirmed avoidance
- Promise ② lossless output: in-pool vs over-pool byte-identical (A2 gate)
- Not promised: never entering the valley (trigger unresolved); general production-grade

What we hope to get back: not "praise", but **leads and verdicts from fellow travelers** — who has seen submit blocking on the driver side, who has reproduced the same slow state, who can help pin the root cause to line level.

Source: pre-release checklist (docs/) §0 and the origin section.

SUMMARY & THANKS

Three sentences

1. The original authors turned “doesn't fit” into “it fits” — KV virtualization lets a consumer card host a million-token workspace;

2. on their foundation we turned “paging” from synchronous waiting into the Async Page Pipeline — movement overlaps compute, over-pool no longer means slowdown;

3. phase 1 delivered, the slow-state manhunt ongoing — goal: from “always fits” to “always fast”.

Thanks again: the KVMem authors (Bilibili: FirePKU) · Shensanshu · tancau (Ring) · Shuaqi-de-zxz (5060Ti line), and llama.cpp / Neroued/ninfer / iamwavecut/ninfer-all and all community contributors.